

8. Cluster Analysis

*“The world is now awash in data
and we can see consumers
in a lot clearer ways.”
— Max Levchin*

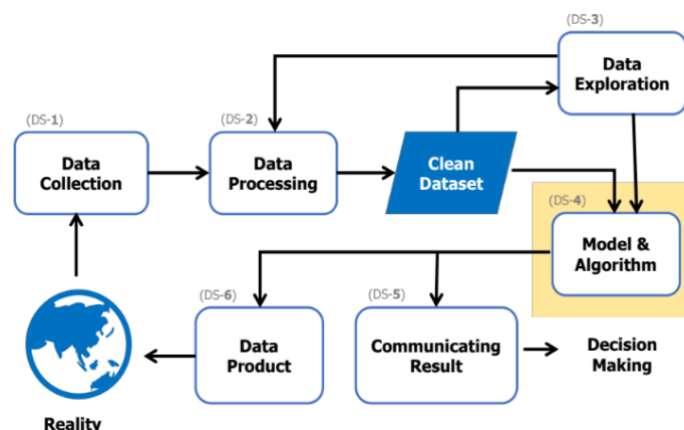


Figure 8.0-1: Cluster analysis in the data science process

Cluster analysis is one of the modeling techniques within the Model Development (DS-4) stage of the data science process (Figure 8.0-1). It is a form of unsupervised learning that groups items by comparing similarity across their features. For example, given customer purchase data with features such as items per order and average price per item, clustering might reveal a group that buys few, expensive items and another that buys many, cheap items — useful for tailoring product recommendations to each group. This chapter covers the principles of cluster analysis, how to evaluate results, and the techniques used.

8.1 Understanding Cluster Analysis

As noted, cluster analysis is unsupervised learning: there are only input features, no target variable — the goal is simply to group the data. Learning here means grouping items with similar features together, and separating items with different features into other groups (Bishop & Nasrabadi, 2006). In Figure 8.1-1, panel (1) shows 16 data points, each with two features, x_1 and x_2 (continuous numbers), plotted with x_1 on the horizontal axis and x_2 on the vertical axis. Real datasets usually have more than two features, which are hard to visualize in higher dimensions, so clustering examples typically use two features for easy 2D visualization.

Panel (2) shows that the data visibly forms two clusters — a darker group on the left and a lighter group on the right — which the human eye can identify directly. Points with similar x_1 and x_2 values sit close together, forming two visible groups; this example uses Euclidean distance to judge closeness.

Judging the closeness and continuity of features this way is central to cluster analysis, so analysts must decide which features to use and which distance measure best fits the problem and data at hand.

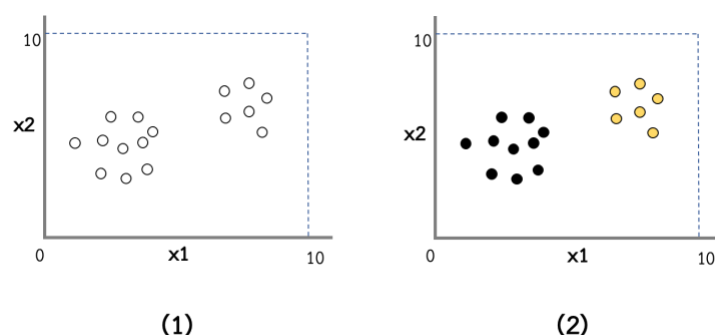


Figure 8.1-1: Example of clustering

8.2 Evaluating Clustering Results

Since cluster analysis is unsupervised — with no target variable to compare against, unlike classification — evaluating its results typically requires a stakeholder or domain expert to judge how realistic the groupings are, based on their own expertise. In practice, clustering results should be presented visually, letting the evaluator spot-check individual members, before the results are trusted enough for production use.

An additional quantitative measure is the Sum of Squared Error (SSE): the total distance between each member and its cluster's representative point. A good clustering keeps members close to their cluster center, yielding a low SSE — useful as a supporting metric alongside expert judgment.

8.3 Clustering Techniques

This book covers two clustering techniques: K-Means and DBSCAN.

K-Means

K-Means assigns members to K centers, where K (the desired number of clusters) is chosen by the analyst before the analysis begins — typically after visualizing the data to estimate a reasonable number of clusters.

Principle

Since this method uses moving centers (centroids) that shift repeatedly until the clustering stabilizes, this book uses the term “boss” as a memorable stand-in for each cluster center: every data point joins its nearest boss, and each boss then moves to the center of its own members, following this algorithm:

1. Randomly place K bosses.
2. Each data point joins the nearest boss, measured by Euclidean distance.
3. Each boss moves to the center of its group — the mean of each feature among its members.
4. Repeat steps 2–3: since the bosses have moved, points may need to switch to a now-closer boss. Continue until the clustering stabilizes — i.e., no member's assigned group changes any further.

Figure 8.3-1 illustrates this with two-feature data and $K=2$, in six steps:

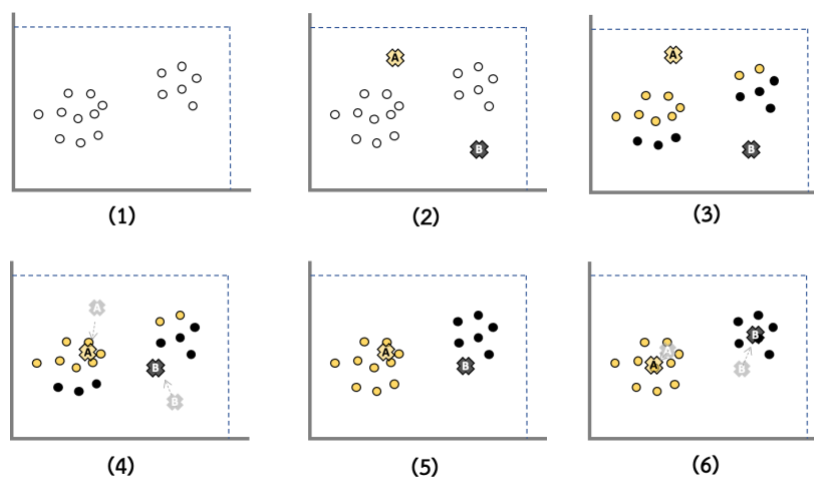


Figure 8.3-1: Example of clustering with K-Means

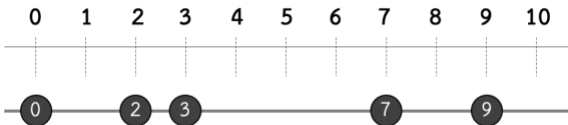
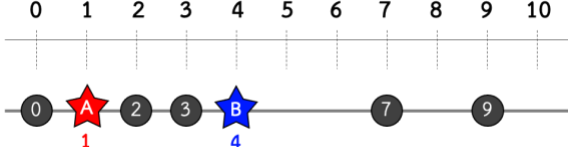
- 1) Show each data point.
- 2) Create 2 bosses ($K=2$), named A and B, at random positions.
- 3) Each member joins its nearest boss, forming an upper group under A and a lower group under B.
- 4) Each boss moves to the center of its current members.

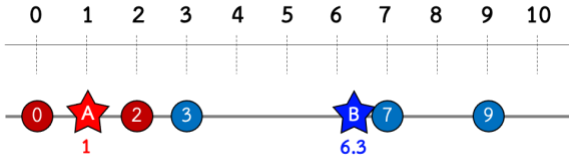
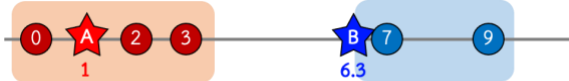
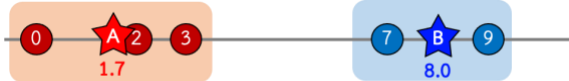
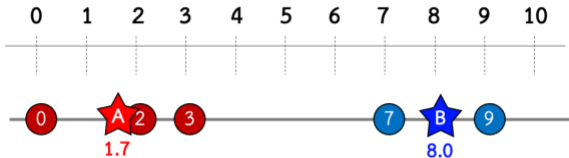
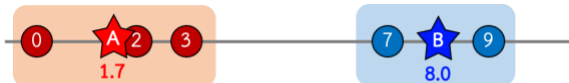
- 5) After the bosses move, each member rejoins whichever boss is now nearest.
- 6) Each boss moves again to its new group's center, settling into two final groups: left and right.

Worked Example

K-Means repeatedly computes each group's center point. Table 8.3-1 walks through this in detail using single-feature data plotted on one axis, to clearly show membership and centroid movement.

Table 8.3-1: Step-by-step walkthrough of K-Means clustering

Round	Step	Clustering action
0 (start)	0.1	Assume 5 data points, each with a single feature, valued 0, 2, 3, 7, and 9. Plotted on the axis: 
	0.2	Apply K-Means with K=2, giving two bosses, A and B: • Boss A (left, red) starts at a random position, 1 • Boss B (right, blue) starts at a random position, 4 
1	1.1	Start of round 1, continuing from the previous state: Boss A = 1, Boss B = 4 
	1.2	Each point joins its nearest boss: • Members 0 and 1 join Boss A • Members 3, 7, and 9 join Boss B 
	1.3	Each boss moves to the center of its members: • Boss A moves to $(0+2) \div 2 = 1.0$ • Boss B moves to $(3+7+9) \div 3 = 6.3$ 

Round	Step	Clustering action
2	2.1	Start of round 2: Boss A = 1, Boss B = 6.3 
	2.2	Each point rejoins its nearest boss: • Members 0, 2, and 3 join Boss A • Members 7 and 9 join Boss B 
	2.3	Each boss moves again: • Boss A moves to $(0+2+3) \div 3 = 1.7$ • Boss B moves to $(7+9) \div 2 = 8.0$ 
3 (final)	3.1	Start of this round: Boss A = 1.7, Boss B = 8.0 
	3.2	Each point rejoins its nearest boss: • Members 0, 2, and 3 join Boss A • Members 7 and 9 join Boss B 
	3.3	Bosses recompute their centers and get the same values as before (A = 1.7, B = 8.0) — membership is now stable (unchanged from the previous round), meeting the stopping condition. Final result: two groups, {0, 2, 3} and {7, 9}. 

Code Example

K-Means can be called from the Scikit-Learn library:

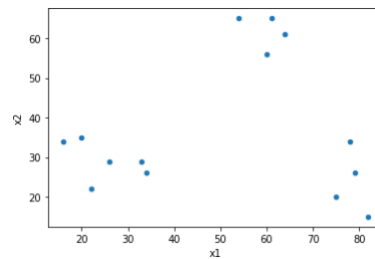
```
import pandas as pd

x1 = [54, 61, 22, 20, 16, 34, 75, 78, 33, 26, 60, 64, 79, 82]
x2 = [65, 65, 22, 35, 34, 26, 20, 34, 29, 29, 56, 61, 26, 15]
df = pd.DataFrame({"x1":x1, "x2":x2})
df
```

	x1	x2
0	54	65
1	61	65
2	22	22
3	20	35
4	16	34

Plotting the data as a scatter plot suggests 3 visible clusters:

```
df.plot.scatter(x="x1", y="x2")
```



```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3)
kmeans.fit(df)
print(kmeans.labels_)

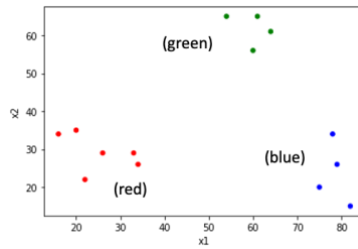
array([2, 2, 0, 0, 0, 0, 1, 1, 0, 0, 2, 2, 1, 1], dtype=int32)
```

```
df["clust"] = kmeans.labels_
df
```

	x1	x2	clust
0	54	65	2
1	61	65	2
2	22	22	0
3	20	35	0
4	16	34	0

Plotting again with cluster labels as colors confirms K-Means grouped the data sensibly:

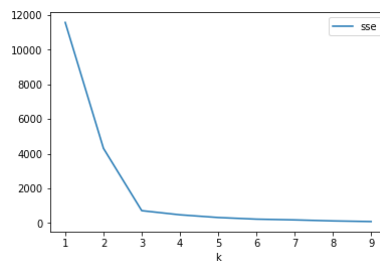
```
colors = ["red", "blue", "green"]
df.plot.scatter(x="x1", y="x2",
               c=df["clust"].astype(int).apply(lambda x: colors[x]))
```



When the ideal K is unclear, try a range of K values (1, 2, 3, ...), record the SSE for each, and plot K against SSE — choosing the K at the “elbow” of the curve:

```
_k = []
_sse = []
for k in range(1, 10):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(df)
    _k.append(k)
    _sse.append(kmeans.inertia_)

sse_df = pd.DataFrame({"k":_k, "sse":_sse})
sse_df.plot(x="k", y="sse")
```



Here, K values below 3 leave members too far from their center, while K values above 3 only slightly improve the result at the cost of smaller, less meaningful groups — making K=3 the sensible choice. For finer analysis, plotting each round's result as a scatter plot can help build intuition further.

DBSCAN

K-Means suits clusters that form compact, roughly circular blobs based on distance from a center, but it struggles with more complex shapes, as in Figure 8.3-2: the expected clustering is shown in (1), but K-Means in practice produces (2) instead. For clusters of arbitrary shape with continuous population density, DBSCAN is a better fit, as explained below.

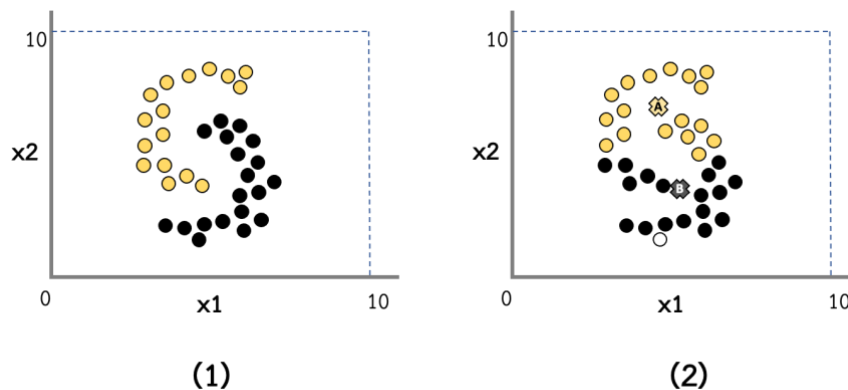


Figure 8.3-2: Example of a more complex clustering shape

Principle

DBSCAN detects cluster shapes based on continuous density (Figure 8.3-3). Think of each point reaching out to “shake hands” with nearby points; if a chain of such handshakes connects continuously, those points form a cluster. Two parameters control this: a radius around each point, and a minimum number of neighbors, both set by the analyst. This produces three types of points:

- Core point — has at least the minimum number of neighbors within its radius.
- Border point — not a core point itself, but a neighbor of a core point.
- Noise point — neither a core point nor a border point; not assigned to any cluster.

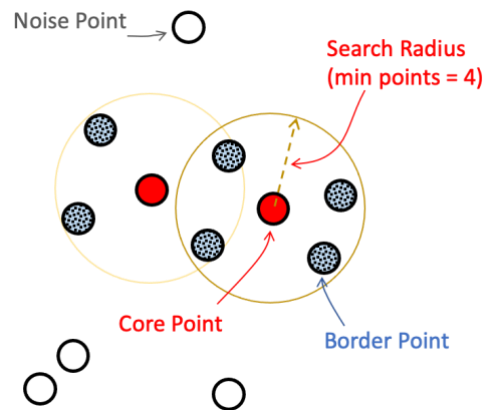


Figure 8.3-3: How DBSCAN works

Once all points are classified, chains of connected core points form clusters, with border points along their edges. Applying this to Figure 8.3-4 (1) with a chosen radius and minimum-neighbor count produces the clustering in (2): dark points belong to a cluster, while light points are unassigned noise. This approach handles complex cluster shapes and reproduces the intended grouping shown earlier in Figure 8.3-2 (1).

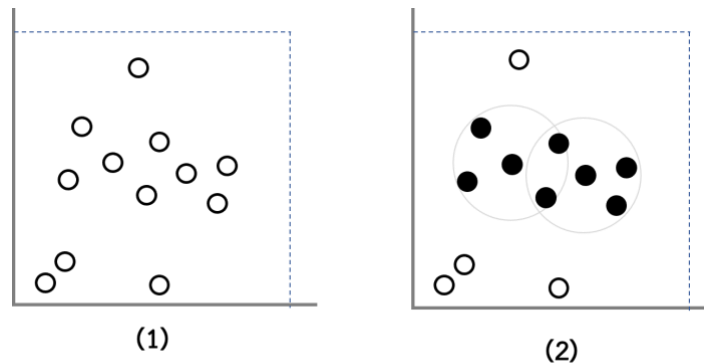


Figure 8.3-4: Example clustering result using DBSCAN

Code Example

DBSCAN can also be called from Scikit-Learn:

```
import pandas as pd

x1 = [54, 61, 22, 20, 16, 34, 75, 78, 33, 26, 60, 64, 79, 82]
x2 = [65, 65, 22, 35, 34, 26, 20, 34, 29, 29, 56, 61, 26, 15]
df = pd.DataFrame({"x1":x1, "x2":x2})

from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=10, min_samples=3)
dbscan.fit(df)
```

```
print(dbscan.labels_)  
  
array([0, 0, 1, 1, 1, 1, 2, 2, 1, 1, 0, 0, 2, 2])
```

8.4 Chapter Summary

This chapter covered cluster analysis, a form of unsupervised learning where the goal is grouping based on features alone, with no target variable. Learning proceeds by grouping members with similar features together. Two techniques were covered: K-Means, suited to compact, blob-shaped clusters, and DBSCAN, suited to clusters of arbitrary shape with continuous member density. Plotting the data as a scatter plot first helps decide which technique fits the data's behavior.

8.5 Review Questions

5. Given 9 data points A–J with features (x1, x2): (-2,2), (0,2), (2,2), (-2,0), (0,0), (2,0), (-2,-2), (0,-2), (2,-2). Using K-Means with K=2 and random starting points (-1,2) and (4,-4), show the calculations and find each cluster's members and center after round 1, round 2, and the final round.
6. Referring to the K-Means code example, write code to view the clustering result for each value of k from 1 to 10.
7. Using data shaped like Figure 8.3-2 (the complex clustering example), write code to build a matching DataFrame, then cluster it with K-Means and display the result as a scatter plot.
8. Using the data created in question 3, write code to cluster it with DBSCAN and display the result as a scatter plot.
9. Discuss whether SSE can be used to evaluate DBSCAN clustering results, and why or why not.

8.6 References

- Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern Recognition and Machine Learning* (Vol. 4, No. 4). Springer.
- Boschetti, A., & Massaron, L. (2015). *Python Data Science Essentials*. Packt Publishing Ltd.
- Igual, L., & Seguí, S. (2017). *Introduction to Data Science*. Springer, Cham.
- O'Neil, C., & Schutt, R. (2013). *Doing Data Science: Straight Talk from the Frontline*. O'Reilly Media, Inc.
- Provost, F., & Fawcett, T. (2013). *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media, Inc.
- Troccoli, E. B., Cerqueira, A. G., Lemos, J. B., & Holz, M. (2022). K-Means Clustering Using Principal Component Analysis to Automate Label Organization in Multi-Attribute Seismic Facies Analysis. *Journal of Applied Geophysics*, 198, 104555.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media, Inc.